ArchSync: Technical Evidence Supplement

ARCHSYNC AUTHOR TEAM

A REPRODUCTION AND INTERPRETATION DETAILS

This appendix makes the development evidence inspectable through operational details for the single-column review profile and a separate supplement for the compact profile. Supplementary-upload acceptance requires confirmation from the venue.

A.1 Keep historical and current evidence separate

Historical D1/D2/P3 observations are reproduced from their pinned benchmark inputs and packaged analyzers. The reporting derivation uses benchmark commit `24d63ebf2fc3075a1d64f1eaff38cdc0b7f586fb`. Its ground truth, Phase 2 and Phase 3 records have individual hashes. Rerunning a current analyzer on that same corpus checks preservation of known development behavior; before/after reporting retains the original observations alongside these later regression results.

The artifact distinguishes original packages, current source snapshots, current distribution builds, first failing runs, repaired runs and unchanged historical result inputs. Current source and package hashes identify the exact evaluated candidate. The accompanying manifest maps each new manuscript count to its raw receipt; installation checks establish operational compatibility.

The historical evidence-location decomposition is explicitly post-hoc. Nine finding-bearing cases concern call sites; cases 08 and 17 concern deterministic anchors for missing edge/path requirements. Case 15 contains two violated rules but counts once. These matches measure the specified locations; completeness, ranking and developer comprehension require separate evaluation.

A.2 Finite oracle construction

For three nodes, six ordered non-self node pairs and two edge types give 12 binary edge slots, hence $2^{12} = 4096$ graphs. For each graph, an adjacency matrix is built independently for all edges and for each individual edge type. Direct requirements query that matrix; path requirements query its transitive closure. The diagonal starts false because a required path must contain an edge; a cycle can establish a nonempty path back to its starting node.

For each of four rule kinds, the source and target independently range over three exact identifiers and a full wildcard, and the edge-type restriction ranges over unrestricted and the two types. Thus $4 \times 4 \times 4 \times 3 = 192$ rule configurations are checked for every graph. The oracle compares per-rule violation counts, aggregate violations and classification. Separately implemented graph-building, selector matching and traversal reduce direct implementation reuse. Shared rule interpretation and developer involvement remain sources of correlated error.

Ordering and consistent-renaming checks use masks 0 through 4080 in steps of 17, giving 241 graph states. Inverse-delta checks compare every graph with the empty-edge graph. They verify that added edges in one direction are removed edges in the other, and that edge changes without rules classify as evolution. Exhaustiveness applies to this declared domain; arbitrary graph pairs, metadata changes, wildcard fragments and source-language programs fall outside it.

A.3 Mutation procedure and retained failures

The mutation harness copies the relevant source and test into a temporary isolated workspace, changes one declared expression at a time, and invokes the same test suite with a fixed timeout. Production source remains unchanged by this procedure. Every run retains the replacement text, original and mutated source hashes, test hash, command output, exit state and native report. A passing unmodified baseline is required before mutant outcomes are interpreted.

Mutant states distinguish test passes, assertion failures and execution errors. The eight faults disable deny enforcement, invert allow logic, invert direct-require satisfaction, ignore path edge types, prevent traversal beyond one edge, miss path target detection, demote violation, or demote evolution. Compile errors and timeouts retain execution-error status. Sensitivity is assessed against these declared faults, with equivalent-mutant analysis outside the campaign's scope.

The original surviving classification mutation exposed a missing assertion on the returned decision despite correct finding counts. Adding that assertion detected the same fault. Both versions remain available, documenting the test-strengthening intervention and its effect on the declared fault model.

A.4 Detector repairs and residual uncertainty

The first adversarial set contains 18 cases: 15 failed before intervention and all 18 passed after it. Positive controls check aliases, captured resources and an unshadowed global fetch alongside local bindings. Type-only import tests deliberately include source that a TypeScript compilation gate would reject, exercising parser robustness when that gate is absent.

A later intervention added eight cases for concatenation and reassignment: seven failed in the cumulative 26-case pre-repair suite. Three further boundary controls were added after repair, yielding 29 passing adversarial cases. Reporting the changing suite sizes preserves the intervention sequence. An old positive fixture concatenated a suffix into an invalid URL port; the fixture was corrected to a valid path suffix and the invalid-port input retained as a negative regression.

A third intervention adds 14 cases to the 29-case suite: 12 write scenarios and two controls for unrelated or shadowed bindings. The identical 43-case suite has 11 failures and 32 passes before the change, then 43 passes afterward. One added write scenario already passed. The repair resolves variable identities within array/object destructuring (including rest/default targets), increment/decrement operations and assignment targets in for-in/for-of loops. For example, an array assignment that overwrites a URL binding must invalidate its initializer as the target of a later fetch. These constructed programs exercise specified behavior.

Recognized-write invalidation is conservative and file-wide, so it can suppress a genuine call before reassignment. Property mutation, imported constructor rebinding, dynamically built templates and alternative fallback destinations remain separate risks. Missing observations remain uncertain even when emitted evidence carries confidence constants. Analyzer version 0.4 invalidates caches from earlier semantics. The full repository gate passed from a clean temporary candidate whose 410 tracked files matched the working source byte-for-byte. The first full run stopped at a CLI assertion still expecting version 0.3; that failure is retained alongside the corrected assertion and passing rerun as packaging and regression evidence.

A.5 Incremental and cache experiment boundary

The incremental suite creates temporary Git repositories and checks additions, modifications, deletions, renames within and between components, and complete-component deletion. Cold and warm results are compared with fresh full scans. The initial nine adversarial cases all failed on the original implementation; a later ignored-source case exposed a further false PASS. Before and after receipts preserve those observations.

Most differential cases compare classification, decisions, finding count and scanned-file count. Cold/warm comparisons additionally inspect architecture deltas and introduced/resolved findings. The nested-component case compares the complete observed graph and evidence. These comparison scopes distinguish aggregate agreement from edge identity and bound the tested Git configurations and filesystem layouts.

The baseline cache separates project-relative roots, architecture identity and analyzer semantics. A payload hash rejects accidentally changed JSON; concurrent writers use distinct temporary publication paths. Cache-writer authentication remains outside this integrity mechanism. Git inventory and scanning require a stable worktree throughout the check. Ignored source outside excluded build/dependency directories is included to match the full scan's source scope.

A.6 Exploratory D3 and independent follow-up

The prepared D3 packet is future exploratory work on 56 primary changes and four context-only changes from three public repositories, using direct cross-group source-file module dependencies. Both author reviewers are development-associated and previously exposed to tool output; their planned source reviews are nonblind, with any AI use recorded and source-verified. Reviews and paired tool runs remain pending; this manuscript reports no D3 results. Source-file endpoints preserve localization detail, while the module-only scope leaves service-level HTTP and database extraction for separate evaluation.

A separate independent follow-up requires a repository frame selected before prediction inspection, lineage-aware development/pilot/holdout separation, and reviewers independent of analyzer development and tuning. They must enumerate supported relations across the complete declared source scope, seal separate labels, and preserve disagreements before adjudication. Full-scope enumeration supplies the missed-relation denominator needed for recall; prediction-only checks address precision. Supported-vocabulary coverage, execution failures and unresolved labels require separate accounting.

A comparator must share a nonempty, frozen observation unit with ArchSync, preserving the distinction between import dependencies and remote service interactions. Performance experiments must pair the same workloads and machines, control the graph-cache condition separately from the operating-system disk cache, separate priming from timed execution and retain failures. These designs specify the proposed follow-up protocol.